NukMiris v0.8.1 — Architecture Challenge, Round 3

Report 1 of 2 — Claude's Responses to Codex

Independent architect response · 28 October 2026 · Design and peer review only. The five original Claude reports are unchanged; this is a new document. References: C1–C5 are my five original reports; 06, 07, 08 are Codex's Round 2 documents.

1. Basis, access and limits

Read in full: 06 (peer review), 07 (decision matrix), 08 (fifteen questions), and the requirements baseline 01 (re-read for the rows the questions turn on: R03, R04, R12, I05, I13, U-NAME, U-GENERIC).

Not available to me: Codex's original architecture documents 02–05. They were not uploaded and I cannot reach the repository from this session. I therefore make **no claim about text I have not seen**. Everything I say about Codex's design comes from 06, 07, 08 and Codex's own self-critique in 06 §E. I also did not read the application source; where I mention baseline code behaviour (for example `booking.js` or `appointment-reminders.js`) I am relying on Codex's descriptions of commit `95decc8`.

Public documentation consulted (read-only, no provider API contacted, nothing installed or executed):

- Google Calendar API error guide — https://developers.google.com/workspace/calendar/api/guides/errors (409 duplicate identifier, 410, 412 `conditionNotMet`)
- Google Calendar `events.list` reference — https://developers.google.com/calendar/v3/reference/events/list (`syncToken` restrictions, `privateExtendedProperty`, `updatedMin`)
- Meta WhatsApp Cloud API webhooks — https://developers.facebook.com/docs/whatsapp/cloud-api/webhooks (retry on non-200 for up to 7 days)
- Meta WhatsApp changelog — https://developers.facebook.com/docs/whatsapp/business-platform/changelog (`biz_opaque_callback_data`, 512-character limit)

- PostgreSQL `btree_gist` — https://www.postgresql.org/docs/current/btree-gist.html (`<>` support for exclusion constraints)

I could **not** retrieve the primary `events.insert` reference. The quotation about undetected ID collisions comes from Codex's reading of the installed Google SDK declaration; I treat it as a credible vendor statement, label it *Codex-reported*, and have not independently verified it.

Evidence labels used in every answer

Label	Meaning
Documented guarantee	Stated in public vendor or PostgreSQL documentation I read
Static deduction	Follows logically from documented behaviour or from the design itself; not tested
Proposed experiment	Would need a separately authorized test (see Report 2 §9)
Owner policy	A product, legal or commercial decision, not an engineering fact
Remaining uncertainty	Unproven by anything I have; stays open

A passing simulator is never used as evidence of provider behaviour, in either direction.

2. Summary of positions

Q	Position	Net change to my design	Main evidence class
1	**Agree**	Remove automatic same-ID insert retry; read-only uncertainty is the default	Documented + uncertainty
2	**Agree**	Armed, nontransferable submission; settle-window resubmission withdrawn	Static deduction

Q	Position	Net change to my design	Main evidence class
3	**Agree**	One unresolved mutation per event; immutable identities; stricter DDL; explicit release evidence	Documented + deduction
4	**Agree**	Withdraw the five-minute advance; restore the ordering barrier; escalate to owner	Owner policy
5	**Agree**	Withdraw my U-GENERIC bounded rule; exact context or baseline history rule	Static deduction
6	**Agree, with new mechanisms**	Deterministic identities; inert replay; gap-recomputed reminders; wider quarantine	Static deduction
7	**Agree**	Archive-before-acknowledge; ingest-time withdrawal marker; pseudonymized evidence ledger; split T-4	Static deduction
8	**Agree**	Serialize withdrawal at ingest; re-check inside the arming transaction	Static deduction
9	**Agree**	Local identity table is the authority; event properties only corroborate	Static deduction
10	**Partly agree**	RLS narrowed to accident containment; hardening list; external break-glass audit	Deduction + experiment
11	**Agree**	Event-aware reschedule check; unfiltered change feed matched by local ID	Documented + deduction
12	**Partly agree**	Three deviations withdrawn; two need owner choice; TTL and transport rules stay	Owner policy
13	**Agree**	Synchronous durability for arming; epochs; version gating; credential-cache rules	Deduction + experiment

Q	Position	Net change to my design	Main evidence class
14	**Partly agree**	Data-class table; crypto-shredding; pseudonymized suppression ledger	Owner policy
15	**Partly agree**	Same-scope slice comparison; my estimate is unmeasured and likely low	Remaining uncertainty

---

Q1. What exactly guarantees same-ID insert safety?

Position: agree. My F-03 and ADR-009 claimed a same-ID retry "either creates once or returns a conflict". The documentation supports only part of that.

What is documented. Google's error guide documents a 409 *duplicate* response, "the requested identifier already exists", for an ID already present in storage **[documented guarantee]**. Codex reports that the SDK declaration also warns that, because the system is globally distributed, collisions may not be detected at creation time **[documented, Codex-reported, unverified by me]**. Together: duplicate detection is best-effort. A random ID protects against *different* creators colliding. It does not make *one creator's repeat* idempotent while the first request may still be in flight.

Failure trace. A's insert times out but remains in flight. B re-inserts the same ID after ten minutes. A then completes. Possible outcomes: B sees a conflict (fine); B succeeds and A's completion is rejected (fine); or both are accepted before detection converges (duplicate or undefined state). No documentation I have excludes the third. **Tombstones:** a deleted event persists as a cancelled resource, so whether and how an ID can be recreated is a provider semantic I cannot assert. **Delete-before-late-insert:** a late original insert could surface after a cancellation. **ID reuse:** my design never reuses an ID after an operation is terminal, so only lost-response retry matters. All three remain **[remaining uncertainty]**.

Would I retain read-only uncertainty until a guarantee exists? Yes. I withdraw P2 and the settle-window insert retry. The corrected mechanism:

1. An uncertain create is reconciled **read-only**: read by event ID and by the operation property. Found and fully verified: applied. Otherwise it stays uncertain and its reservation stays held.
2. The reconciler keeps polling on a widening schedule (for example 1 min, 5 min, 30 min, 2 h, daily for several days) so a late-appearing event is adopted rather than missed.

3. An audited operator decision may *accept residual risk*, release the slot hold and mark the operation `abandoned_unverified`. If the event appears later it is flagged as a late orphan. This is a risk acceptance, not a proof **[owner policy]**.
4. A sentinel counts events sharing one operation property or one deterministic ID. More than one is an incident.

Cost I accept: a slot with an uncertain create stays blocked until evidence or an operator decision. Frequency and blocked-slot cost are measured in E1 and E9.

What E1 can and cannot show. Sandbox runs can *characterize* observed behaviour. They cannot prove the absence of a delayed collision. If Google gives a contractual statement, I would revisit.

Amend: C1 F-03 and S-1; C2 §8.3 step 6, §8.4 rows 4–5, §9.2 P2/P3, the settle-window paragraph; C3 ADR-009 (decision and "benefits"); C4 scenario 3 (Insert row).

Q2. How is a paused external dispatcher fenced?

Position: agree. A SQL fence rejects a stale commit *after* the provider call has already been made. C2 §4.3 protects commits, not remote effects, and C4's "lease and fence plus unique key prevent a second attempt" overstated it.

The permission I should have defined. Submission needs a **nontransferable armed-attempt permission**:

- **D1. One dispatcher component per process.** Every provider client lives behind it; a lint rule forbids provider clients elsewhere. Each process start registers a fresh `dispatcher_id`. Multiple loops in one process share that single dispatcher, so they cannot each hold a permission.
- **D2. Arming.** A transaction verifies fence, lease and `prepared` state plus every local gate (§Q8), then sets `armed`, `armed_by = dispatcher_id`, `armed_at`. No socket write is allowed before that commit returns.
- **D3. Nontransferable.** Once `armed`, only `armed_by` may submit. The reclaim query selects `prepared` rows only; lease expiry never moves an armed attempt.
- **D4. Stale owner.** An armed attempt whose dispatcher stops heartbeating is accounted as `uncertain`. No other dispatcher may submit *anything* for that attempt or operation. If the original dispatcher resumes, it may complete **at most the one request it already owns**, after a conditional `armed → dispatching` update that fails if an operator has since superseded the attempt.

- **D5. Protection outlives the process.** An unresolved armed attempt keeps its reservation, its block on later same-event mutations and its place in the review queue regardless of lease or instance death.
- **D6. Provider-side bounding.** Absolute HTTP deadlines; template TTL for Meta; `If-Match` for Google patch and delete.

Re-running the counterexample. A commits the marker at fence 7 and pauses. B claims fence 8, sees `armed_by = A`, and does **nothing**: no retry, no re-schedule, no release. A resumes and sends once. The request is late, but it is the *only* request, and the reservation protecting its interval is still held. The counterexample is closed **only because the settle-window retry is withdrawn** (Q1).

When may another dispatcher submit? Only a *new* attempt, and only after the previous attempt is terminal by evidence: applied, definitely rejected, never armed, or an audited operator decision (the U-RETRY path).

Residual windows I cannot close.

- A process frozen between the final conditional update and the socket write (VM freeze, `SIGSTOP`, a partitioned host not known dead). The effect is bounded to one request on a protected interval. For reminders, provider TTL bounds how late it can be delivered, but R03's 15-minute rule is not enforceable once a process is frozen **[remaining uncertainty]**.
- Old hosts: see Q6 for credential-gated dispatch, which bounds an old host's ability to send at all.

```
stateDiagram-v2
  [*] --> Prepared
  Prepared --> Armed: arming tx commits, fence checked, local gates pass
  Armed --> Dispatching: conditional update by armed_by only
  Dispatching --> Applied: verified evidence
  Dispatching --> Rejected: definite refusal
  Dispatching --> Uncertain: timeout, 5xx, crash, dispatcher lost
  Armed --> Uncertain: dispatcher lost before the socket write
  Uncertain --> Applied: later evidence
  Uncertain --> Closed: operator decision, new linked attempt
```

Evidence: static deduction. **Proposed experiment:** E3. **Amend:** C2 §4.2–4.3, §8.3–8.4, §10.1, §11.3; C3 ADR-004 and ADR-008; C4 scenarios 4 and 5.

Q3. What completes the exclusion constraint's proof?

Position: agree. The constraint is correct for what it says and **documented**: `btree_gist` supports `<>` for exclusion constraints **[documented guarantee]**. But when two operations share one appointment key, `appointment_key <>` is false, so both reservations are admitted. It is a backstop for *different* appointments, not a proof of serialized mutation of one event.

Completion, as an illustrative design sketch (not executed, not a schema change):

```
-- Illustrative only. Never run in this exercise.
CREATE TABLE appointment_identities (
  business_id uuid NOT NULL, calendar_id uuid NOT NULL,
  appointment_key text NOT NULL,            -- provider event id; see Q6 for
derivation
  customer_id uuid NOT NULL, created_by_operation uuid NOT NULL,
  PRIMARY KEY (business_id, calendar_id, appointment_key),
  FOREIGN KEY (business_id, customer_id) REFERENCES customers (business_id,
id));

CREATE TABLE operations (
  id uuid PRIMARY KEY, business_id uuid NOT NULL, calendar_id uuid NOT NULL,
  appointment_key text NOT NULL,            -- set at admission, immutable
  kind text NOT NULL CHECK (kind IN ('book','move','cancel','attend')),
  state text NOT NULL,
  FOREIGN KEY (business_id, calendar_id, appointment_key)
    REFERENCES appointment_identities);

-- At most one unresolved mutation of any one event, attendance and cancel
included.
CREATE UNIQUE INDEX one_unresolved_mutation ON operations
  (business_id, calendar_id, appointment_key)
  WHERE state IN ('admitted','claimed','armed','uncertain');

CREATE TABLE calendar_reservations (
  calendar_id uuid NOT NULL, appointment_key text NOT NULL,
  operation_id uuid NOT NULL REFERENCES operations,
  during tstzrange NOT NULL CHECK (
    NOT isempty(during) AND NOT lower_inf(during) AND NOT upper_inf(during)
    AND lower_inc(during) AND NOT upper_inc(during)),
```

```
  state text NOT NULL CHECK (state IN ('held','committed','released')),
  EXCLUDE USING gist (calendar_id WITH =, during WITH &&, appointment_key
WITH <>)
    WHERE (state IN ('held','committed')));
```

What each part closes.

- **Two mutations of one event** (move, cancel, attend, in any pair): the partial unique index admits only one operation in the unresolved set. Entry to the set is a single insert or a row-locked state transition. Uncertain operations stay in the set, so a later cancel cannot race an unresolved move.
- **Caller-supplied keys:** the key comes from the identity row via a composite foreign key and is immutable. Customers never supply it (§Q9).
- **NULL, empty and unbounded ranges:** `NOT NULL` columns and the `CHECK` make them inexpressible. Buffer is included in `during` from the business configuration version recorded on the operation.
- **Rollback:** a rolled-back transaction loses a tentative row but no provider request, because the arming transaction (Q2) must commit before any request exists.

Which expired protections can be released.

Protection	Release rule
`held`, operation never armed	After the hold lifetime, by a conditional update on a volatile predicate evaluated in the statement, not in the constraint
`held` or `committed`, operation armed or uncertain	Never by time. Only by evidence: positive proof the write applied (then it transitions), a recorded definite provider refusal, or an audited operator decision
`committed`, appointment cancelled	On verified cancel

What evidence permits releasing an uncertain write? For a **Calendar create** or a **WhatsApp send**, no automatic evidence of non-application exists in anything I can cite, so the answer is *operator risk acceptance only* (Q1). For a **conditional Calendar mutation** there is a stronger option. Google documents a 412 when the `If-Match` ETag no longer matches the resource **[documented guarantee]**. If we read the event, see that our change is absent, and then **bump the ETag ourselves** with a harmless conditional write (for example a property update carrying the old ETag), any older in-flight mutation

carrying that old ETag can no longer apply. If our bump returns 412, the stale request applied first and we learn it. This turns uncertainty into proof for patch and delete **[static deduction; depends on Google evaluating preconditions atomically, experiment E1]**. It does not exist for creates.

Remaining: local constraints never fence remote calls or manual Google writers. Codex's union-and-transfer alternative is equally unproven until it has DDL; I would test both in E2.

Amend: C2 §8.1–8.2, §12.1; C3 ADR-008; S-10 scope.

Q4. Why is five-minute uncertain-order advancement acceptable?

Position: agree. It is not. I13 concerns *definitely unsent* expired prompts. An uncertain attempt is different: it may still be delivered. Letting later replies overtake it after five minutes changes I05, and I presented it as a clarification (F-05). It was a proposed product change.

The failure. Earlier send A is uncertain. After five minutes B is released and delivered. A then arrives, carrying a stale prompt or instruction after B's newer one. Server-side offer validation means A's buttons are inert (**safety holds**), but the customer can read contradictory text (**experience does not**).

Corrected position.

1. Preserve the barrier. An uncertain attempt blocks later *ordinary* replies in that conversation until resolved by evidence, expired by definite non-submission, or released by an audited operator decision. Reminders remain a separate stream (F-07).
2. Distinguish three things: an **unsent expired prompt** (dropped, never blocks); an **uncertain attempt** (blocks); a **terminally rejected** attempt (does not block).
3. Surface the cost: oldest blocked conversation age is an alert and a review item.

An honest limit. Meta can keep trying to deliver an accepted message for a long period, so *strict* ordering could in principle block a conversation for a long time. Neither a pure barrier nor a timeout is satisfactory without a product decision. I would **request a changed customer contract only if** blocked-conversation duration measured in E4 and E9 is unacceptable; options for the owner are (a) pure barrier with operator release, (b) a long, owner-approved maximum hold, (c) self-contained, version-stamped replies that tolerate late stale predecessors **[owner policy]**.

Evidence: owner policy and static deduction. **Amend:** C1 F-05; C2 §10.1 "Order" paragraph, §9.2; C3 ADR-010; C4 scenario 6.

Q5. How can generic confirmation safely ignore old requests?

Position: agree. My U-GENERIC rule ("exactly one current candidate") is wrong. Counterexample, accepted: reminder A for the old schedule is delivered late after a move; B (the new schedule) is the sole *current* request; the customer's unquoted "Sí" answers A. My rule would confirm B. The baseline's R12 says delayed or stale context must never retarget, and its U-GENERIC row says old visible-message ambiguity is not discarded merely because time has passed.

Why "exactly one current candidate" does not identify intent. Current-ness is a property of the *system's* schedule, not of the *customer's* reading of their phone. A visible earlier message remains a plausible referent.

Correction. Revert to the baseline rule:

1. **Quoted or button context** binds to the exact dispatched attempt, then occurrence, then current schedule. If the schedule has changed, the action is invalid and the customer gets recovery guidance. It never falls back to another appointment.
2. **Unquoted generic text** acts only when exactly **one** dispatched request (accepted or uncertain) exists across the customer's retained history for that business, including requests on cancelled, moved or past schedules, with no age cutoff. Otherwise the customer is asked to use the specific reminder button.
3. Consequence: after any move there are at least two retained requests, so a generic reply is ambiguous and falls to the button prompt. That is the baseline's intended behaviour.

Cost. More button prompts. A bounded history policy would need a *demonstrated* safe rule and owner approval **[owner policy]**.

Evidence: static deduction. **Proposed experiment:** E5 (stale unquoted confirmation after a move). **Amend:** C1 U-GENERIC row; C2 §11.4; C4 scenario 8 item 4 (consistent, no change); C3 ADR-011 text.

Q6. How are lost attempt and operation identities reconstructed during restore?

Position: agree with the gap; I add mechanisms that narrow it but do not close it. A raw inbound body contains none of the identifiers allocated *after* it arrived: the operation ID, the Calendar event ID, the outbound attempt ID. Replay with fresh randomness would

allocate new ones, and a new ID defeats every identity-based protection. No receipt exists for a send that left after the snapshot. Codex is also right that quarantine must cover Calendar writes and old instances, not just sends.

M1. Deterministic identities, for discovery (not for retry). Derive identifiers from stable inputs and a per-business key held in KMS, not in the database: `operation_id = MAC(k_biz, "op" | provider_event_key | offer_id | ordinal)`; `event_id = base32hex(MAC(k_biz, "evt" | operation_id))` at 128 bits or more; `attempt_id = MAC(k_biz, "att" | source | effect_ordinal)`. After database loss, replay recomputes the same IDs and **reads** the event by ID instead of inserting. It does *not* rely on insert deduplication (Q1). This is a design choice; whether a MAC-derived ID is as safe as the UUID that Google recommends is my reading and is unverified **[static deduction; remaining uncertainty on provider side]**.

M2. Inert replay. Archived inbound events are reprocessed in a mode where effect execution is disabled: no Calendar write, no send, no reminder dispatch. The reducer still computes state and emits a *discovered-effects list* for review. Many replays are naturally inert: a selection token refers to a server-side offer that lived in the lost database, so it resolves to ExpiredView. An offer created *before* the restore point survives, so a post-snapshot selection can be reprocessed. Its operation ID is deterministic, so the executor reads by ID and either adopts what it finds or holds. It never creates.

M3. Reminders: recompute the gap instead of enumerating lost attempts. Reminder occurrences are a deterministic function of appointments and times. Every occurrence whose `due_at` falls in the gap, from the restore point minus a margin to quarantine exit, is marked `gap_uncertain` and never sent. Occurrences already more than 15 minutes late at exit are skipped by R03 anyway. So the duplicate-reminder bound does not depend on finding the lost attempt rows **[static deduction]**. It needs a gap start (the restore point) and an upper bound for the failure time. If the failure time is unknown, the bound is quarantine exit.

M4. Ordinary replies. Replies that replay would regenerate are suppressed. A customer may receive no reply and can write again. That is a deliberate preference for silence over a repeated actionable message, and I withdraw my earlier claim that duplicate ordinary replies are "not harmful" (a confirmation or stale choice can mislead). It is a behaviour the owner should approve **[owner policy]**.

M5. Discovering unknown scopes.

- **Calendar writes:** a per-business sweep of the change feed since the restore point minus a margin, matched against local identities (Q11). Events that carry NukMiris properties but have no local identity become orphan review items. This gives real evidence of affected scopes, using documented reads.
- **WhatsApp sends:** no provider history is relied on. Unknown sends cannot be enumerated, so *sends* stay quarantined for every business until archive completeness is established (Q7) and the gap-recomputation in M3 is applied.

M6. Quarantine scope and old instances. Quarantine pauses **all provider calls**, not only sends. The single external dispatch authority is the **credentials themselves**: at restore, revoke the old environment's KMS decrypt grants and rotate Meta tokens (Google refresh tokens are re-probed, and revoked where an old environment might still hold them). Cached tokens expire within a short, fixed cache lifetime (for example five minutes), and egress from the old environment is denied. An old instance not known to be dead can then only complete requests whose credentials it already holds in memory, for that lifetime **[static deduction; experiment E3/E6]**.

Remaining. Sends made after the last archived record and with no receipt cannot be listed. Their affected scope may therefore stay quarantined longer than anyone wants. That is the real cost of not trusting an incomplete record, and it is why T-4 must be split (Q7).

Amend: C2 §16.3 steps 2 and 4–7, §16.4; C3 ADR-018 (add deterministic identity and inert replay); C4 scenario 10 table; add ADR-023.

Q7. What atomically preserves withdrawals and operator mutations outside the failed database?

Position: agree. C2 §5.1 acknowledges after the database commit and exports to the archive on a best-effort basis afterwards. A STOP that is acknowledged, then lost with the primary before export, exists nowhere. Adding an operator command journal is another dual write with the same hole. My earlier claim of "zero lost STOPs" was unjustified.

B1. Archive-before-acknowledge for inbound. `web` acknowledges only when **both** the database commit and a conditional write of the raw body to the archive have succeeded. If either fails it returns a non-200. Meta documents retrying non-200 responses with decreasing frequency for up to seven days **[documented guarantee]**, so Meta's own

queue holds the event through a short outage of either store. A duplicate archive write is idempotent by object key. Cost: acknowledgement latency and availability become coupled to the object store (measured in E9).

B2. Withdrawal evidence ledger. A small append-only record (business, pseudonymized sender, type, time, source event key) is written in the same ingest transaction *and* to the archive before acknowledgement. It is deliberately minimal so it can outlive the raw payload (Q14).

B3. Operator mutations. The operator command path writes a journal entry to the archive **first** (intent), commits the database change, then writes a commit marker; the operator is acknowledged after the marker. After a restore, entries with an intent but no marker are *ambiguous* and go to review, with an **asymmetric safe default**: an ambiguous *withdrawal* is applied (suppress), an ambiguous *grant* is not applied. Losing a grant under-sends; losing a withdrawal over-sends.

B4. Completeness and gap detection. Archive objects are named by channel and provider event key, and carry a per-channel ingest sequence number. After a restore, compare the archive set with the database set after the restore point and look for sequence gaps. Because acknowledgement required the archive write, an acknowledged event cannot be missing unless the archive itself lost data, which a daily manifest hash, versioning and retention lock are meant to detect.

B5. Suppression on missing evidence. If archive completeness cannot be verified (store unreachable, anomaly, manifest mismatch), consent-gated sending (reminders) stays suppressed for every customer until it is. Ordinary service conversation is not consent-gated and may resume.

Can T-4's two-hour write resumption coexist with zero lost withdrawals? Only conditionally. If B1–B4 operate and the manifest check passes, yes. Otherwise reminders remain blocked and "zero lost withdrawals" is not claimable. So **T-4 splits**: **T-4a** inbound acceptance and read-only service; **T-4b** booking and management writes, per business, after the Calendar sweep; **T-4c** reminder sending, after evidence completeness is verified. These remain proposals **[owner policy]**.

Remaining. Simultaneous loss of the database and the archive (they must sit in separate failure domains and, ideally, separate cloud accounts). Compromise of the archive store.

Amend: C2 §5.1 steps 4–6, §16.1–16.3, the operator command journal paragraph; C3 ADR-018; C1 T-4; C4 scenario 10; add ADR-024.

Q8. How does permission withdrawal win during slow preflight?

Position: agree. C2 §11.3 checks permission early in a sequence that includes slow Google and template reads. A STOP can be durably accepted while the worker is waiting. An inbox row that the mailbox has not yet processed does not suppress that worker.

Correction: two changes, both serializing at the last local step.

1. **Withdrawal fast path at ingest.** The ingest transaction applies the same pure recognition function as the reducer to detect an exact withdrawal command and, in the *same transaction* as the event row, upserts a suppression marker for that business and pseudonymized sender. This is *suppress-only*: if the reducer later decides the text was not a withdrawal, an audited step lifts the marker. The baseline says opt-out keeps its stated priority (W05) and gives the supported phrases; the exact set and how it interacts with protected name entry are owner/baseline decisions I list in Q12.
2. **Final admission inside the arming transaction** (Q2 D2). After all slow reads, the arming transaction re-reads, under `FOR SHARE` on the suppression row: suppression marker, effective permission, occurrence state, schedule identity, reminder mode, business configuration version, credential version and status, and template row status and contract hash.

Linearization. If the ingest commit precedes the arming transaction's snapshot, arming sees the marker and refuses. If arming commits first, the STOP is linearized *after* that armed attempt, which may still be delivered because a sent message cannot be recalled. **A withdrawal therefore always wins over unarmed work, never over armed work.** That is the strongest claim available.

Remote changes during slow checks. A template paused, or a token revoked, *at the provider* after our check cannot be serialized locally. I expect the provider to reject the request (a definite refusal), but that is unverified **[remaining uncertainty; E4, S-8]**. Locally, a credential revocation flips a version checked in the arming transaction, and cached tokens are evicted on the next arm. A TTL or template-contract change is a local row read at arming against R03's transport-before-start budget.

Not solved: a manual Calendar change in the few seconds between the preflight read and arming (a state-of-world race).

Evidence: static deduction. **Proposed experiment:** E5. **Amend:** C2 §5.1 (add the fast path), §11.3 steps 2–6 and §11.5; C4 scenario 5 table; C3 ADR-011.

Q9. What is the complete customer and event ownership predicate?

Position: agree. `nm_biz` alone is not an authorization boundary. Private extended properties can be edited or copied by anyone with write access to the Calendar.

The authority is local. A customer never supplies an event ID. Their text is never parsed for one. Every management action begins from an opaque offer token that resolves to a **local appointment identity** already bound to that conversation, business and customer. So "customer A supplies customer B's event ID" has no input path.

Mandatory checks at every Calendar effect, all on trusted local state:

1. The offer or action token is valid for the authenticated conversation, business and customer (L5).
2. The conversation's `customer_id` equals the identity row's `customer_id`.
3. `(business_id, calendar_id, appointment_key)` exists in `appointment_identities` and the `calendar_id` equals the business's *current mapping revision*.
4. The operation row is the unique unresolved mutation for that identity (Q3).
5. The Calendar read uses the Calendar and credential from the business row, never from the token. The event ID equals the local key; ETag, times and schedule identity match the local expectation.
6. Properties (`nm_biz`, `nm_cust`, `nm_op`, `nm_sched`) are **corroboration only**. A mismatch moves the appointment to review and stops sends, matching R08's "invalid owner enters review".

Copied or forged properties. A copied event has a different ID and therefore no local identity: it is unmanaged and, at most, an orphan review item. An owner who edits properties on a real event triggers the mismatch path. Properties are not cryptographic proof against a malicious owner, which is outside the threat model.

Lookup correction. My §7.4 listed a customer's appointments from a Google filter on `nm_cust`. Lookup must instead start from local identities for that customer and verify each against Google; filter results that have no local identity are excluded and reviewed.

Evidence: static deduction; baseline `appointment-management.js` already checks a fuller tuple, per Codex. **Amend:** C1 F-04; C2 §7.2 ("managed by NukMiris" sentence), §7.4, §12.2 L5/L6; C4 scenario 7 rows 3, 7 and 8.

Q10. What prevents selecting arbitrary RLS tenant context?

Position: partly agree. Yes: `app_runtime` can run `SET LOCAL app.business_id = <B>` in my design. Custom settings are not authenticated, so my claim that application code "cannot forget a `WHERE business_id`" was fine but ADR-012's broader framing was too strong.

What RLS protects against here. *Accident containment*: a forgotten or wrong predicate, a bad join, a reporting query, an operator tool that omits scope, a repository function used under the wrong context. **What it does not protect against:** a compromised or malicious runtime that can run arbitrary SQL (it can set any tenant), SQL injection with that privilege, or a privileged role. I withdraw any wording that implies otherwise. A real boundary against a compromised shared runtime would need per-tenant credentials and connections, which I consider too costly for this scale. The blast radius of a compromised `worker` is therefore **all tenants**, and I will say so **[static deduction]**.

Hardening list (each a proposed test, E7):

- **Definer functions:** owned by a non-login role with only the needed `SELECT`; fixed `search_path` set in the definition; no dynamic SQL; `REVOKE ALL FROM PUBLIC` then `GRANT EXECUTE` to named roles; a test that reads `pg_proc` ACLs and fails on `PUBLIC` execute. Discovery functions return identifiers only and verify state.
- **Pooling reset tests:** under transaction pooling `SET LOCAL` is transaction-scoped; ban session-level `SET`; a generated test interleaves tenants on shared connections and asserts the setting is unset at each transaction start.
- **Multitenant raw batches:** a raw body may hold events for several channels. Store raw batches in an ingress-only schema no tenant role can read; tenant tables hold only that tenant's extracted event. Raw archive access is separately restricted.
- **Per-business operator grants:** an operator-to-business grant table checked by the admin API before any tenant context is established, with a reason on each action. The database cannot authenticate a human, so this is application-enforced and audited; RLS is secondary.
- **Break-glass audit that the role cannot disable:** issue the role through short-lived cloud IAM database authentication behind an approval step; ship database audit logging (for example `pgaudit`) to an **external, append-only store in a separate account**; make the audit settings superuser-only so the role cannot turn them off; alert on any break-glass session. A database-internal audit table is not trustworthy against a role that bypasses restrictions.

Evidence: static deduction; the exact privileges are an experiment, E7. **Amend:** C2 §12.2 L3 and L4 wording and the roles table, §13.2; C3 ADR-012 (narrow the claim); C4 scenario 7 rows 4–5.

Q11. How are reschedule availability and synchronization complete?

Position: agree on both points. Free/busy returns merged busy intervals. If the moving event overlaps another busy event, removing the moving event's old interval can hide the other event. And my filtered `updatedMin` sync assumes deletion records and edited events remain discoverable by the property filter.

Event-aware target check (reschedule).

1. Compute the target interval including buffer, and the moving event's own interval.
2. If the two do not overlap, a free/busy query over the target is sufficient.
3. If they overlap, list events over the target window **at event level** (single events, restricted fields, every page), exclude **exactly** the moving event by ID with a matching ETag, and evaluate each remaining event under the busy semantics fixed in S-4 (cancelled, transparent/free, declined and all-day cases). Where semantics are ambiguous, **block**.
4. Any page failure, timeout or malformed page is `Unavailable`, never free.
5. Other NukMiris appointments are already arbitrated by the database constraint; this check is for non-NukMiris events and for lag between Google and our records.

Change feed, corrected. Google documents that a sync-token request cannot be combined with `privateExtendedProperty`, `timeMin`, `timeMax`, `updatedMin` and several other parameters, and that an expired token returns 410 and requires a full resync **[documented guarantee]**. It also documents that deleted entries since the last list are always included **[documented guarantee]**. So the filtered design I wrote cannot use a sync token, and a property filter could never find an event whose property was removed.

- **Primary sync:** an **unfiltered** sync-token feed with a minimal field set. Each change is matched by the **local appointment key** (the ID is the one thing that identifies a deletion record; I do not rely on a deletion record carrying any property).
- Changed events that are unknown locally but carry NukMiris properties become orphan review items. A known event whose property was removed or altered goes to review (R08).

- **Independent safety net:** a daily direct read by ID of every local active appointment, plus the preflight read before each reminder. Detection never depends on the feed being complete.
- **Expired token or any 410:** full resync, bounded by a time window on the initial request.
- **Privacy note:** the feed returns other events' times; fields are restricted to what availability already needs.

Pagination and cursor advancement. A sync run is a recorded object: `(run_id, started_at, page_token, pending_next_sync_token)`. Each page is processed in **one transaction** that applies its upserts and advances the stored page token. The new sync token is adopted only when the final page's transaction commits. A crash resumes from the stored page token or restarts from the old token; upserts are idempotent. A rejected page token also triggers a restart from the old sync token.

Evidence: documented guarantees for the constraints above; static deduction for the rest. **Proposed experiment:** E8 (merged busy events during a move; all-day and transparent events; property removed; tombstones; pagination and token rewind). **Amend:** C2 §7.3 (final recheck bullet), §7.4, §7.5; C3 ADR-007 and ADR-014; S-3 and S-4 scope.

Q12. Which deviations from baseline conversation and reminder policy are intentional?

Position: partly agree. Of the four deviations Codex lists, I did not intend any of them as settled behaviour. I presented several as clarifications when they were proposals. My corrected default is **baseline parity unless the owner approves a change**.

Deviation	My answer	Owner approval needed?	Acceptance cases that would change
Only `Menu` overrides name entry (C2 §6.4)	**Withdrawn.** Implement the baseline: literal navigation commands can override protected entry, and multiword names that merely *contain* those words remain names	Only if the owner later wants stricter command collision handling	None under parity
Poison-event skipping after three failures (C2 §5.4)	**Withdrawn as a default.** It weakens I05. Default is fail-closed: the conversation stays blocked, an alert fires, and a *reviewed* operator skip is available. Which event classes could ever be skippable is a product question	Yes, for any automatic skip	Add “poison event parks the conversation”; keep submitted-operation continuations unskippable

Deviation	My answer	Owner approval needed?	Acceptance cases that would change
Expired name input routed through Main (C4 scenario 8 vs C2 §6.1)	**My documents conflict.** I propose the conservative reading: an expired name prompt gives recovery guidance only, creates **no customer and no operation**, and the text is not re-interpreted as a command. Whether current behaviour also lets the same message act as an ordinary command needs checking against the baseline	Yes, if it differs from current behaviour	Expired-prompt cases
Waiting for a late parent receipt before skipping a follow-up (C2 §11.3 step 5)	**Withdrawn as default.** The baseline (R04) requires matching delivered or read evidence; per Codex, current behaviour skips at first processing. A bounded wait within the allowance is an option, not parity	Yes	Follow-up timing cases; adds a late-receipt case

Deviation	My answer	Owner approval needed?	Acceptance cases that would change
TTL caps and transport-before-start budget	**Confirmed, and my text was too weak.** I said TTL is "template metadata checked at readiness". R03 requires respecting the TTL and transport budget at send time. The arming gate (Q2 D2) must verify, using the baseline's constants, that the send respects the due-through-15-minutes window, the appointment start, and the kind's TTL cap (Codex cites 900 s and 300 s) plus the transport allowance	No (parity)	None; add a boundary test at the arming gate

Two further items for the owner, from Q8 and Q4: the exact withdrawal phrase set and its interaction with protected name entry, and the ordering-barrier policy.

Q13. What durable commitments survive HA failover and deployment rollback?

Position: agree. "Every step is restartable" is true of unarmed work and insufficient for armed effects. If the arming transaction commits on the old primary and a remote request is issued, a failover that loses that commit lets a replacement see `prepared` work and arm it again.

Dispatch admission contract.

1. **Durability before dispatch.** The arming transaction runs with synchronous commit to a standby (or the managed service's equivalent). A dispatcher may call a provider only after that commit is acknowledged under that contract. If no synchronous standby is

available, commits block or the dispatcher refuses to arm, so the system fails *stopped*.

2. **Epochs.** A `cluster_epoch` row is incremented at every promotion. Arming records the epoch; a dispatcher refuses to arm if its cached epoch differs from the current one.
3. **Post-failover reconciliation.** After promotion, dispatch is paused. Every attempt armed in an earlier epoch is accounted `uncertain`, and a Calendar sweep runs per business before dispatch resumes. If the failover was degraded (asynchronous, possible lost commits), treat it as a disaster-recovery event (Q6, Q7), not routine.
4. **Quiescence for deploys.** Draining means: stop arming; finish only owned armed attempts; wait up to the HTTP deadline; then exit. A forced kill leaves armed attempts uncertain, which is accepted.
5. **Version compatibility.** Armed rows carry a format version. `min_dispatcher_version` is checked in the arming transaction, so an older image cannot arm under newer rules. Migrations follow expand-then-contract and never change the meaning of an existing armed row. A rollback is permitted only to an image whose supported armed-format range includes every armed row.
6. **Rolling overlap.** Old and new dispatchers may both arm. Correctness does not depend on version agreement beyond item 5.
7. **Credential cache.** Cache lifetime is short and fixed. Arming checks the credential version and status. A revocation bumps the version, so no dispatcher arms with a stale credential. A paused process holding an already-fetched token remains the Q2 residual window.

What I cannot assert. What a specific managed Postgres offering actually guarantees in each failover mode (S-12, E3). Whether lag on a read replica or a delayed promotion can resurrect an old state. These need the chosen service's documentation and a destructive test.

Evidence: static deduction. **Amend:** C2 §4.3, §14 (deployment row), §16; C3 ADR-004, ADR-016; C4 scenario 4 and the failover row of scenario 11; add ADR-022.

Q14. How do privacy and recovery obligations coexist?

Position: partly agree. I proposed an immutable raw archive without stating what it holds, how long, or how a person's data leaves it. That needs a policy. The proposals below are engineering defaults for the owner and counsel to accept or change; **I claim no legal compliance**.

Data class	Contents	Custody	Proposed retention	Erasure and key policy
Raw inbound archive	Provider payloads including message text and sender	Write-once store in a separate cloud account; encrypted under a per-business data key	30 days **[owner policy]**	**Crypto-shredding**: destroy the business key when retention ends or the business offboards. Lock mode that allows a documented two-person override for legal orders, not compliance-mode locks that cannot be broken
Withdrawal and consent evidence ledger	Business, **keyed-MAC pseudonym** of the sender, type, time, source event key. **No message text**	Same store plus the database	As long as the consent obligation requires **[owner policy]**	Not erased while the obligation exists; pseudonymized so it can honour an opt-out without keeping the raw number
Operator command journal	Command type, scoped identifiers, reason, operator identity	Write-once store	Audit period set by the owner	Pseudonymize customer references; no message text
Database operational data	Names, phone identifiers, appointments, operations	Managed database	Business-defined	Erasure on request; copies in backups age out with backup retention (35 days proposed). State that period in the privacy notice

Data class	Contents	Custody	Proposed retention	Erasure and key policy
Calendar copies	Names and phones inside the business's own events	**The business's Google account**	Business-controlled	NukMiris cannot erase these without the business authorizing a change. An erasure request is relayed to the business; automated deletion is outside v0.8.1 **[owner policy]**
Credentials	Ciphertext bound to business, provider and credential ID	Database	Until rotation grace ends	Key versions retired by schedule; key deletion uses a waiting period

Two different threats, separated.

- **Stolen database or backup dump:** credential ciphertext is useless without KMS access. But names and phone numbers are *not* application-encrypted in my design; managed disk encryption protects against media theft, not against a stolen logical dump. Column-level protection of identifiers is an option with query-cost trade-offs.
- **Runtime compromise holding KMS access:** the attacker can decrypt every tenant's tokens within that grant. Per-business keys give audit granularity, not isolation. Controls are alerts on decrypt volume, short caches and fast revocation. The blast radius is all tenants (Q10).

Lost revocations are handled by Q7. **Remaining uncertainty:** jurisdictional requirements, retention minima for consent evidence, and whether pseudonymized evidence satisfies them.

Amend: C1 F-12 and T-7; C2 §13.3, §16.1; C3 ADR-013 and ADR-018; add ADR-024 (retention and custody).

Q15. What evidence makes a rebuild preferable at equal scope and cost?

Position: partly agree. Codex is right that my seven-to-ten-month greenfield estimate and its eight-to-fourteen-week selective estimate are unmeasured and describe different scopes. I also did not read the source, so Codex holds evidence about the existing implementation that I lack; my C5 preference for a rebuild was a design-first judgment, not an assessment of code.

Comparison protocol (I adopt 07's, with my additions).

1. **Same gates for A, B and C:** parity (the requirement-tagged fault matrix), security (E7), recovery (E6), and the Round 3 safety contract in Report 2.
2. **Same vertical slice** under B and C: signed acceptance, new-customer selection and name, protected booking, lost response and restart, read-only reconciliation, atomic reminders and output, exact attendance, destructive-loss restore.
3. **Metrics:** (a) illegal outcomes under the shared fault matrix, which must be zero to continue; (b) *evidence quality*, meaning whether each failure leaves proof or only logs; (c) a representative **change exercise** (for example add a reminder kind; change the service-window policy), measured in files touched, tests added and reviewer time; (d) an **operator incident exercise** (seeded uncertain send plus restore gap, diagnosed from the runbook); (e) full **cost**: engineer time to reach the gates, managed database HA, containers, KMS, secrets, locked storage, telemetry, a canary WhatsApp number, and on-call hours; (f) **measured workload** from a defined burst model.

Which result makes me choose selective rearchitecture (B)? If the slice shows that the existing pure logic and tests (date grammar, intent rules, availability, slot labels, reminder schedule, permission rules) port or stay with small change, **and** the existing execution paths can be given the armed-attempt, identity and quarantine semantics through a small number of seams without two active authorities, then B is better. I would also choose B if the change exercise costs no more than about 1.25 times the fresh slice. These thresholds are my proposals, not facts.

What would make me choose C? If satisfying D1–D6, archive-before-ack, deterministic identities and the unresolved-mutation constraint requires replacing persistence, dispatch, delivery and execution together, then the seams are the rewrite and C is cheaper and clearer.

What would invalidate my assumptions.

- **10× headroom:** a burst model (Monday-morning reminder ticks, hot tenants), claim contention on a single conversation, Google read budget, and the added latency of archive-before-ack. It must be replaced by observations.
- **Seven to ten engineer-months:** the Round 3 mechanisms (dispatcher protocol, archive-before-ack, evidence ledger, deterministic identities, event-aware sync) add work. My honest revision is **roughly two to four more engineer-months for the same gates, so about nine to fourteen**, still a judgment and still unmeasured. Spike failures (S-1, S-6, S-10) would move it further.
- **Small-team on-call:** invalid if page volume in the canary exceeds what one rotation of two people can absorb, if provider-standing work dominates, or if only one engineer exists. Fallback for a pilot: a stated business-hours support commitment.

Evidence: remaining uncertainty by design; it is resolved only by the slice. **Amend:** C5 §7 (estimate and sequence), C5 §9.

---

3. Amendment register (original passage → change)

Originals stay unedited. These are the changes Report 2 carries into the combined design.

Original passage	Change
C1 F-03, S-1	Same-ID insert retry is not assumed; S-1 must characterize collisions, tombstones and late completion; read-only default
C1 F-04	Ownership is local identity first; properties corroborate
C1 F-05; C2 §10.1 order; C3 ADR-010	Five-minute advance withdrawn; barrier restored; owner decision
C1 U-GENERIC; C2 §11.4	Baseline history rule or exact context
C1 U-NAME/U-COMMAND; C2 §6.4	Baseline command override restored
C1 T-4	Split into T-4a, T-4b, T-4c
C1 S-2 fallback	No read-compare-write downgrade; stop mutations instead
C2 §4.3; C3 ADR-004	Armed-attempt, nontransferable submission protocol (D1–D6)

Original passage	Change
C2 §5.1; C3 ADR-018	Archive-before-acknowledge; ingest-time withdrawal marker
C2 §5.4	Poison events fail closed
C2 §7.2, §7.4	Local identity authority; lookup starts from local rows
C2 §7.3, §7.5; ADR-007, ADR-014	Event-aware reschedule check; unfiltered change feed matched by local key; daily direct reads; atomic page and cursor handling
C2 §8.2; ADR-008	Identities, one unresolved mutation per event, stricter DDL, release-evidence rules
C2 §8.3–8.4, §9.2; ADR-009	Settle-window insert retry removed; etag-bump fence for conditional mutations
C2 §11.3, §11.5	Arming-transaction final admission; TTL and transport gate at send
C2 §12.2; ADR-012	RLS reframed as accident containment; hardening list; external break-glass audit
C2 §14, §16; ADR-016, ADR-018	Synchronous durability for arming; epochs; quarantine covers all provider calls; credential-gated dispatch; deterministic identities; inert replay; gap-recomputed reminders
C5 §7	Estimates replaced by the comparison protocol; revised range labelled a judgment